


國立台灣海洋大學資訊工程學系 109 學年度第 1 學期計算機組織小考四

學號

姓名

1. MIPS 指令 lw, beq 用到 ALU 算術邏輯單元的動作分別是加法與減法, 其 ALUop 編碼分別是 00 與 01。 slt 用到 ALUop 編碼是 10, 代表是個 R-format 指令, 其 ALU 算術邏輯單元的動作是 slt(若小於則設定)。(35%)
2. slt 的控制單元輸出向量 [RegDst, ALUSrc, MemoReg, RegWrite, MemRead, MemWrite, Branch, ALUop1, ALUop0] 是 100100010, lw 的 rt, 8(rs) 的控制單元輸出向量 [RegDst, ALUSrc, MemoReg, RegWrite, MemRead, MemWrite, Branch, ALUop1, ALUop0] 是 011110000; lwr, rd, rt(rs) 的控制單元輸出向量 [RegDst, ALUSrc, MemoReg, RegWrite, MemRead, MemWrite, Branch, ALUop1, ALUop0] 是 101110010。(30%)
3. 最簡單的 MIPS 處理器架構, 一個指令分成 IF 指令讀取 ID 指令解碼 EX 執行 MEM 記憶體存取 WB 寫回暫存器 等五個階段/週期完成。(10%)
4. 提高處理器效能最常見的方法是使用管線/流水線/pipeline 技術, 這是利用重疊/overlap 不同指令執行時程來做到, 這種技術可以改善處理量/吞吐量/throughput, 但無法改善指令完成(所需)時間/執行延遲/completion time or latency。(20%)
5. 妨礙指令在下一週期進入處理器管線執行的情況叫作危險(hazard), 這些情況更可進一步區分成三類: 結構(structural)危險、數據/資料(data)危險、控制(control)危險。(20%)
6. Data Hazard 主要有那三種解決方式: 指令順序調動/code reordering or scheduling、前饋繞送/forwarding bypassing、管線停滯/pipeline stall。(15%)
7. Control Hazard 主要有那三種解決方式: 指令順序調動或延遲分支/code reordering or delayed branch、分支預測/branch prediction、管線停滯/pipeline stall。三者之中要為達到高效能管線處理器運作, 會使用分支預測/branch prediction 技術。(20%)
8. 將 MIPS 處理器由無管線化設計, 轉成管線化設計, 基本上要增加那些硬體, 請繪製簡圖表示。(20%)



5. 妨礙指令在下一週期進入處理器管線執行的情況叫作危險(hazard)，這些情況更可進一步區分成三類：結構(structural)危險、數據/資料(data)危險、控制(control)危險。(20%)⁴
6. Data Hazard 主要有那三種解決方式：指令順序調動/code reordering or scheduling、前饋繞送/forwarding bypassing、管線停滯/pipeline stall。(15%)⁴
7. Control Hazard 主要有那三種解決方式：指令順序調動或延遲分支/code reordering or delayed branch、分支預測/branch prediction、管線停滯/pipeline stall。三者之中要為達到高效能管線處理器運作，會使用分支預測/branch prediction技術。(20%)⁴
8. 將 MIPS 處理器由無管線化設計，轉成管線化設計，基本上要增加那些硬體，請繪製簡圖表示。(20%)⁴

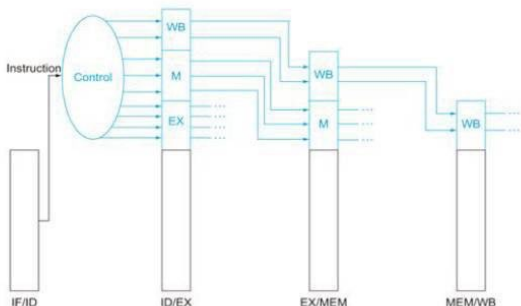


圖4.50 最後三級的控制訊號線



9. 前饋單元(Forwarding Unit)的簡化之輸入偵測條件是那 4 點? 輸出是那 2 個信號, 這 2 個信號如何運作? (40%)

1a. EX/MEM.RegisterRd = ID/EX.RegisterRs

Fwd from
EX/MEM
pipeline reg

1b. EX/MEM.RegisterRd = ID/EX.RegisterRt

2a. MEM/WB.RegisterRd = ID/EX.RegisterRs

Fwd from
MEM/WB
pipeline reg

2b. MEM/WB.RegisterRd = ID/EX.RegisterRt

多工器控制	來源	解釋
ForwardA=00	ID/EX	ALU 的第一個運算元來自暫存器檔案
ForwardA=10	EX/MEM	ALU 的第一個運算元前饋自前一個 ALU 運算結果
ForwardA=01	MEM/WB	ALU 的第一個運算元前饋自數據記憶體或再早的 ALU 運算結果
ForwardB=00	ID/EX	ALU 的第二個運算元來自暫存器檔案
ForwardB=10	EX/MEM	ALU 的第二個運算元前饋自前一個 ALU 運算結果
ForwardB=01	MEM/WB	ALU 的第二個運算元前饋自數據記憶體或再早的 ALU 運算結果

10. 停滯單元(Stall Unit or Hazard Detection Unit)的簡化之輸入偵測條件是那 2 點? 主要輸出是什麼信號, 會造成什麼效果? (20%)

```
if (ID/EX.MemRead and
    ((ID/EX.RegisterRt = IF/ID.RegisterRs) or
     (ID/EX.RegisterRt = IF/ID.RegisterRt)))
    stall the pipeline(停滯管道)
```

Force control values in ID/EX register to 0

- EX, MEM and WB do nop (no-operation)

Prevent update of PC and IF/ID register



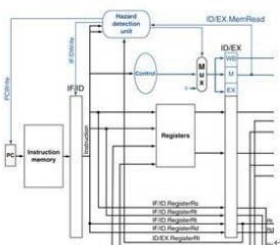
10. 停滯單元(Stall Unit or Hazard Detection Unit)的簡化之輸入偵測條件是那 2 點? 主要輸出是什麼信號, 會造成什麼效果?(20%)

```
if (ID/EX.MemRead and
    ((ID/EX.RegisterRt = IF/ID.RegisterRs) or
     (ID/EX.RegisterRt = IF/ID.RegisterRt)))
    stall the pipeline(停滯管道)
```

Force control values in ID/EX register to 0

- EX, MEM and WB do **nop** (no-operation)

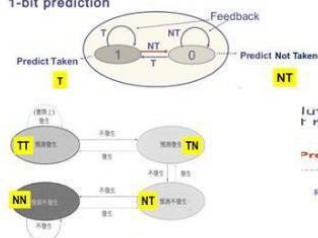
Prevent update of PC and IF/ID register



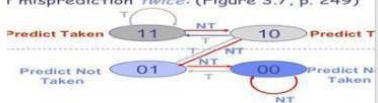
11. 請圖示 1-bit 分支預測器與 2-bit 分支預測器的工作原理, 兩者主要的差別是什麼? 若一個

分支連續執行的結果是 T F T F T T T F T T T T，請問 1-bit 與 2-bit 分支預測器(初始狀態分別是 T 與 TT)預測準確率分別是多少？(40%)

1-bit prediction



Evolution: 2-bit scheme where change prediction on misprediction twice: (Figure 3.7, p. 249)



2-bit 分支預測器連續預測錯誤兩次才改變分支預測方向，1-bit 分支預測器預測錯誤一次就改變分支預測方向。

Branch Outcomes:	T	F	T	F	T	T	T	F	T	T	T	T
1-bit predictor (Initial state: T)	Y	N	N	N	N	Y	Y	N	N	Y	Y	Y
2-bit predictor (Initial state: TT)	Y	N	Y	N	Y	Y	Y	N	Y	Y	Y	Y

預測準確率: 1-bit predictor 7/13; 2-bit predictor 10/13

12. 現代化進階處理器，主要用那兩個方法來進一步提升單一處理器效能：增加管線深度(deeper pipelining, superpipelining)與多重派發指令(multiple issue)。(10%)



2-bit 分支預測器連續預測錯誤兩次才改變分支預測方向，1-bit 分支預測器預測錯誤一次就改變分支預測方向。

Branch Outcomes:	T	F	T	F	T	T	T	T	F	T	T	T
1-bit predictor (Initial state: T)	Y	N	N	N	N	Y	Y	Y	N	N	Y	Y
2-bit predictor (Initial state: TT)	Y	N	Y	N	Y	Y	Y	Y	N	Y	Y	Y

預測準確率：1 bit predictor 7/13; 2 bit predictor 10/13

12. 現代化進階處理器，主要用那兩個方法來進一步提升單一處理器效能：增加管線深度(deeper pipelining, superpipelining)與多重派發指令(multiple issue)。(10%)
13. 指令多重派發(Instruction Multiple Issue)已經是現代化進階處理器的標配，我們現在常用智慧型手機處理器用到的 ARM A78 處理器核心，每個時脈週期可以最多派發 6 個指令，而 2020 年問世的 Apple A14 與 M1 處理器晶片之中用到的 Firestorm 核心，以及 ARM 最新發表的 Cortex-X1 核心，每個時脈週期可以最多派發 8 個指令。(10%)
14. 你對目前計算機組織課程與教授的內容有任何疑問困難？意見或建議？(10%)